

Reasoning in LLMs

图文讲义版：用原 PPT 作为视觉锚点，按“概念故事线 + 直觉解释 + 考试速记”整理。

Reasoning as structured textual deliberation → Training-time vs inference-time improvement → Chain-of-Thought as one linear scratchpad → Self-consistency as sampling plus majority vote → Tree of Thoughts as explicit search over states → Self-reflection as feedback-driven revision → External verifiers/solvers as tool-grounded checking → Limits: cost, shared bias, bad feedback, wrong formalisation

What is Reasoning?

ChatGPT says: To reason is to think in a structured way, using facts or assumptions to draw conclusions, make decisions, or explain why something follows from something else.

We will focus on the notion of using “text” (chain-of-thought, for example) to argue about/solve something that humans typically indeed need reasoning for

这节课一句话

这节课的核心不是把 LLM 神秘化成“会思考”，而是把 inference-time reasoning 拆成三种可操作的脚手架：多走几条路再投票、把解题过程变成搜索、让模型或外部工具检查并修正推理。

1. 先把 reasoning 放回地面：它是一种可检查的文字草稿

What is Reasoning?

ChatGPT says: To reason is to think in a structured way, using facts or assumptions to draw conclusions, make decisions, or explain why something follows from something else.

We will focus on the notion of using “text” (chain-of-thought, for example) to argue about/solve something that humans typically indeed need reasoning for

人话直觉

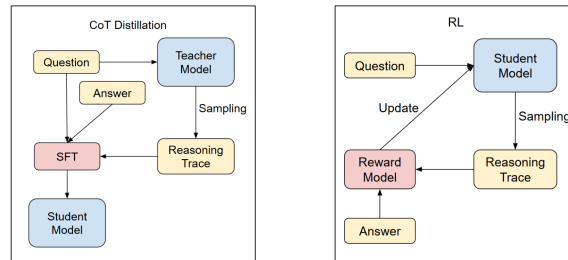
不要一上来就把 reasoning 想成模型脑子里真的有一个小人在思考。更安全的理解是：我们给模型一张“草稿纸”，让它把事实、假设、中间结论写出来。草稿不保证正确，但它让推理变得可采样、可比较、可批改、可交给工具检查。

精确定义 / 机制：讲义采用的 reasoning 定义是 structured thinking：使用 facts 或 assumptions 得出 conclusions、decisions，或解释为什么 something follows from something else。本讲进一步收窄到 text-based reasoning，典型形式是 chain-of-thought：模型不只输出 final answer，而是生成一段 intermediate reasoning trace。这个 framing 很重要，因为后面的方法几乎都不改模型参数，而是操控推理 trace 的生成、选择、评价或验证。

考试问法：如果问“本讲如何理解 reasoning”，先给结构化思考定义，再强调本讲关注 text/CoT 形式；最后补一句：reasoning trace 是推理辅助和评估对象，不等于天然正确的证明。

2. 为什么本讲偏爱 inference-time: 不重训，也能给模型加脚手架

Reasoning - training



(Thanks to Shiwen Qin for figure)

- Two “training” approaches mostly (does the one on the left look familiar?)
- We will focus today on improving inference (easier within limited-budget settings)

人话直觉

训练模型像重新培养一个学生：成本高、周期长。Inference-time 方法像考试现场给学生更好的草稿策略：多做几遍、列分支、检查答案、调用工具。它不一定让学生本质上更聪明，但常常能让当场表现更稳。

精确定义 / 机制：讲义区分 training-time 和 inference/decoding-time 改进。Training approaches 包括 CoT distillation、RL 等，会通过数据或 reward 改变模型行为；本讲重点是 decoding phase 的技巧，因为 limited-budget settings 下更容易尝试。可考方法包括 stepwise reasoning (CoT、self-consistency、Tree of Thoughts)、self-reflection，以及 external solvers/verifiers。它们共同点是主要增加推理时计算、采样、prompting、搜索或工具接口，而不是重新训练模型。

考试问法：如果问“inference-time reasoning 为什么有吸引力”，答：不改参数、实现相对简单、适合预算有限场景；代价是推理成本、延迟和方法设计复杂度会上升。

3. Self-consistency: 不要迷信第一条 CoT, 让多条草稿投票

Self-Consistency

- The idea: there are different ways to reason about a problem. If several reasoning paths lead to the same answer, choose that answer
- How do we action that?
- Sample a set of reasoning chains $r_1, \dots, r_n$ with answers $a_1, \dots, a_n$
- Choose the answer a which repeats the most in the sequence a_i

人话直觉

做数学题时，一个人第一次写的草稿可能误入歧途；如果他用不同思路做五遍，其中三遍都得到同一个答案，我们会更愿意相信这个答案。Self-consistency 就是把这种“多路独立草稿 + 最终投票”搬到 LLM 上。

精确定义 / 机制：Self-consistency 的机制是采样多个 reasoning chains $r_1, \dots, r_n$ ，每条链产生一个 answer $a_1, \dots, a_n$ ，然后选择出现次数最多的 answer a 。它不是让一条 CoT 更长，而是利用 decoding stochasticity 产生 diverse reasoning paths。最简单的多样性控制手段是 temperature：temperature 较高会带来更多探索，但过高会引入噪声。该方法适合答案可聚合的任务，如数学题、多选题、常识推理；风险是如果模型偏差共享，很多路径可能以不同话术走向同一个错答案。

$$r_1, \dots, r_n \rightarrow a_1, \dots, a_n, \quad \hat{a} = \operatorname{argmax}_a \sum_{i=1}^n \mathbf{1}[a_i = a]$$

考试问法：高频问法是“self-consistency 是什么、如何操作、优缺点”。答题必须包含 sample multiple reasoning chains、majority vote、temperature 控制多样性、成本增加和多数不保证正确。

4. Tree of Thoughts: 把一条线性 CoT 改造成搜索树

Tree of thoughts

Instantiating ToT requires answering:

- Defining thought decomposition
- Thought generator - given a state in the tree, generate a new thought. For example: (a) sample from the LLM; (b) use a propose prompt
- State evaluator - given a frontier of states, evaluate the progress they make towards solving the problem (just like in search). For example: (a) value the state based on a prompt; (b) vote for the best state (so comparison among different states)
- Search algorithm - Breadth-first search (BFS); Depth-first search (DFS)

Note that ToT is problem specific, less generic, need to instantiate it

人话直觉

CoT 像从迷宫入口一路往前走，走错了也可能硬着头皮编下去。Tree of Thoughts 更像真正解题：先提出几个中间想法，看哪些有希望，再扩展、比较、必要时回退。它把“想法”变成搜索节点，把解题变成 search problem。

精确定义 / 机制：ToT 针对 CoT 的线性局限。要实例化 ToT，讲义列出四个组件：thought decomposition 定义一个 thought 的粒度，既要小到能产生多样性，又要大到能评价进展；thought generator 在当前 state 下生成候选 thought，可通过 LLM sampling 或 propose prompt；state evaluator 评价 frontier states 离目标有多近，可用 value prompt 或 vote prompt；search algorithm 决定如何探索，例如 BFS 或 DFS。ToT 更 problem-specific，因为不同任务需要不同 thought 粒度、评价标准和搜索策略。

考试问法：如果考 Tree of Thoughts，不要只说“树状 CoT”。要列出四件套：decomposition、generator、evaluator、search algorithm，并解释它允许 exploration、self-evaluation 和 backtracking；同时指出成本高、任务定制强。

5. Self-reflection: 模型先写答案，再给自己可执行的批改意见

Self-Reflection

Originally, self-refinement? (Madaan et al., 2023)

- The idea: Let the model “refine” its answer by providing *feedback* on its own answer
- Can do this in multiple steps
- Leads to a form of reasoning, or “self-reflection”

人话直觉

Self-reflection 像让同一个学生换一支红笔批改自己的答案：先写一版，再指出哪里不好，然后重写。它有用的前提不是“模型会内省”，而是反馈真的能定位问题、推动下一版变好。

精确定义 / 机制：Self-reflection 或 self-refinement 的基本流程是：生成初始输出，生成 feedback，再根据 feedback refine 输出，可重复多轮。停止条件可以是固定 timestep t ，也可以从反馈中抽取 stopping indicator 或 scalar stop score。它适合写作、代码、开放式回答等可迭代改进任务，但并非总能提升：如果模型看不出自己的错误，或者反馈空泛甚至错误，迭代会把答案改坏。

$$y^{(0)} = M(x), \quad f^{(k)} = M(x, y^{(k)}), \quad y^{(k+1)} = M(x, y^{(k)}, f^{(k)})$$

考试问法：如果问 self-reflection，答出 generate → feedback → refine → repeat/stop；再比较 self-consistency：前者围绕一个答案迭代修订，后者采样多条链并投票。

6. Feedback 质量: actionable + specific 才能让反思变成改进

About Feedback (Madaan et al., 2023)

- The model needs to be prompted to write feedback that is *actionable* and *specific* (via $fb^{(k)}$)
- Actionable: concrete action that would improve the output; Specific: identify concrete phrases to change

Example for code reasoning:

- Good: *Avoid repeated calculations in the for loop*
- Bad: *Improve the efficiency of the code*

人话直觉

“再优化一下”这种反馈听起来像建议，其实没有方向；“for loop 里别重复计算”才像真正能下手的批注。Self-reflection 的瓶颈常常不在 refine 这一步，而在 feedback 是否具体到能执行。

精确定义 / 机制： Madaan et al. 强调模型需要被 prompt 成生成 actionable and specific feedback。Actionable 指反馈包含能改进输出的具体行动；specific 指反馈定位到具体短语、代码片段、推理步骤或错误位置。讲义中的代码例子非常典型：Good feedback 是 Avoid repeated calculations in the for loop；Bad feedback 是 Improve the efficiency of the code。前者给出可执行修复方向，后者只是泛泛评价。

考试问法：若问“为什么 feedback prompt 重要”，答：没有 prompt 约束时模型容易给空泛评价；高质量 feedback 必须具体、可执行，才能指导下一轮输出，而不是制造貌似合理的自我安慰。

7. External verifiers/solvers: 让模型别只自证，找外部裁判

Using External Verifiers/Solvers

- The idea: use an external tool to solve a problem/question
- Breaks the boundaries of LLM-only modelling, but is a decoding-only method in most cases

人话直觉

如果模型像学生，external verifier 就像计算器、编译器、证明检查器或监考老师。学生可以写出思路，但某些步骤最好交给更硬的系统验证。这样突破了 LLM-only modelling，但仍常发生在 inference 阶段。

精确定义 / 机制： External solvers/verifiers 的核心思想是在回答过程中调用外部工具解决或检查问题。它通常仍是 decoding-only method，因为模型参数不变，只是在推理时接入工具。Lean 是讲义中的数学推理例子：Lean 通过 type checking 验证形式化证明是否符合规则。关键 caveat 是 verifier 检查的是 formalisation，而不是原始自然语言本身；如果 LLM 把 prompt p 转成 Lean 时就错了，即使 type checks，也不保证原题答案正确。

考试问法：如果考 external verifiers，答：优点是提供比模型自评更独立、更可靠的检查信号；难点是 natural language 到 formal language 的转换、工具接口、搜索空间，以及错误 formalisation 会让验证结果失去意义。

8. 把三类方法放在一张考场对照表里

Summary

Reasoning in LLMs often refers to a process of deliberation that the LLM follows to reach an answer. We covered:

- Stepwise: chain-of-thought, self-consistency, tree-of-thoughts...
- Self-reflection
- Using external solvers

人话直觉

复习时别把名词背散。把它们想成三种“当场增强”：self-consistency 是多份草稿投票，ToT 是搜索树规划，self-reflection 是自我批改，external verifier 是找外部裁判。共同目标是让答案不只靠一次贪心生成。

精确定义 / 机制：本讲总结为 stepwise methods、self-reflection、external solvers。CoT 是基础线性 trace；self-consistency 在完整 trace 层面做 sampling and voting；ToT 在中间 state 层面做 branching search and evaluation；self-reflection 在输出层面做 feedback-driven iteration；external solvers/verifiers 把部分判断交给工具。所有方法都要记住 tradeoff：更多推理时计算可能换来更好表现，但不会自动消除模型偏差、错误反馈、错误 formalisation 或任务不匹配。

考试问法：综合题常问比较。推荐作答顺序：先定义 CoT baseline，再说 self-consistency 多链投票，ToT 显式搜索和回退，self-reflection 反馈修订，external verifier 工具检查；最后用一句 tradeoff 收尾。

考试速记版

- Reasoning in this lecture = structured text-based deliberation, often via CoT, not a guarantee of truth.
- 本讲主轴是 inference-time/decoding-time improvement：不重训，靠采样、搜索、反馈、工具增强推理。
- CoT 是单条线性 reasoning path；走错后可能沿错误方向继续。
- Self-consistency = sample multiple reasoning chains and majority-vote final answers.
- Temperature 是控制 self-consistency diversity 的最简单旋钮；太低没多样性，太高噪声大。
- ToT = thought decomposition + generator + evaluator + search algorithm (BFS/DFS).

- ToT 比 self-consistency 更显式地评价中间状态，允许 backtracking，但更 expensive/problem-specific。
- Self-reflection/self-refinement = initial output → feedback → refined output，可多轮迭代。
- Good feedback must be actionable and specific; “Improve efficiency”太空，“Avoid repeated calculations in the for loop”才能改。
- External verifiers/solvers 在 inference 时调用工具，突破 LLM-only modelling，但依赖正确 formalisation。
- Lean type checking 能验证形式化证明规则，不等于自动解决自然语言题，也不保证 formalisation 正确。
- Slides 25-29 的 Lean mini-course 细节按 exam QA 标注不考；可考重点是 verifier 思想和错误类型。

一段稳妥考场回答

LLM reasoning 可以理解为模型通过文本化的 deliberation，例如 chain-of-thought，组织事实、假设和中间步骤来得到答案。本讲主要关注 inference-time 方法，因为它们不需要重新训练模型，而是在 decoding 阶段通过额外计算提升表现。最基础的 CoT 只生成一条线性推理链；self-consistency 会采样多条 reasoning chains $r_1, \dots, r_n$ ，并对最终答案 $a_1, \dots, a_n$ 做多数投票，优点是简单通用，缺点是成本更高且多数路径可能共享同一错误偏差。Tree of Thoughts 进一步把推理变成搜索：先定义 thought decomposition，再用 generator 产生候选 thought，用 evaluator 评价 frontier states，最后用 BFS/DFS 等策略探索，因此支持 self-evaluation 和 backtracking，但更 problem-specific。Self-reflection 则让模型生成答案、给出 actionable and specific feedback、再迭代 refine，停止于固定步数或 stop signal；它依赖反馈质量。External verifiers/solvers 如 Lean 把部分检查交给外部工具，能提供更强的正确性信号，但自然语言到 formalisation 的转换可能出错，所以 type check 通过不必然说明原题答案正确。